

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1.Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

ANSWER:

INTRODUCTION:

A **network topology** describes how computers and networking devices are arranged and connected in a network. One of the commonly used network topologies is **Star Topology**.

In a star topology, all computers or hosts are connected to a **single central networking device**, usually a switch. The central switch controls communication between the connected hosts.

In this program, Python is used to generate a star topology for a given number of hosts. Each host is connected to the central switch using a **Cat6 UTP cable**, and the cable length is displayed.

2. Definition of Star Topology

Star topology is a network topology in which every host has a separate point-to-point connection to a **central switch or hub**.

Here:

- **H1–H5** → Network hosts
- **SW1** → Central switch
- **Lines** → Cat6 UTP cables
- **10 m** → Cable length

The arrangement looks like a **star**, which is why it is called Star Topology.

CODE:

```
# Star Topology Generator
```

```
def generate_star_topology(num_hosts):  
  
    switch_name = "SW1"  
  
    cable_length = 10  
  
    print("\nStar Topology with", num_hosts, "Host(s) connected to",  
switch_name)  
  
    print("=" * 70)  
  
    print("Host\tConnected Switch\tCable Type\tLength (m)")  
  
    print("-" * 70)  
  
    for i in range(1, num_hosts + 1):  
  
        host = "H" + str(i)
```

```

print(host, "\t", switch_name, "\t\tCat6 UTP\t", cable_length)

print("=" * 70)

print("Verification: All", num_hosts, "host(s) are connected to", switch_name)

print("All communication between hosts occurs through the central switch.")

n = int(input("Enter the number of hosts: "))

generate_star_topology(n)

```

CODE OUTPUT:

```

Enter the number of hosts: 5

Star Topology with 5 Host(s) connected to SW1
=====
===
Host      Connected Switch  Cable Type  Length (m)
-----
---
H1      SW1      Cat6 UTP    10
H2      SW1      Cat6 UTP    10
H3      SW1      Cat6 UTP    10
H4      SW1      Cat6 UTP    10
H5      SW1      Cat6 UTP    10
=====
===
Verification: All 5 host(s) are connected to SW1
All communication between hosts occurs through the central switch.

Code Execution Successful

```

Components Used in the Diagram

A. Hosts

A **host** is a computer or network device that communicates over the network.

In our diagram:

- H1 = Host 1
- H2 = Host 2
- H3 = Host 3
- H4 = Host 4
- H5 = Host 5

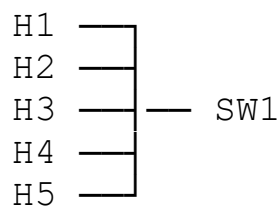
The program can generate any number of hosts based on the user's input.

B. Central Switch – SW1

SW1 represents the central Ethernet switch.

The switch is the most important device in this topology because all hosts are connected to it.

For example:



When H1 wants to communicate with H3:

H1 → SW1 → H3

The switch receives the frame from H1 and forwards it toward H3.

C. Cat6 UTP Cable

Cat6 UTP stands for **Category 6 Unshielded Twisted Pair** cable.

It is used as the physical transmission medium between each host and the switch.

In our diagram:

H1 — Cat6 UTP — SW1

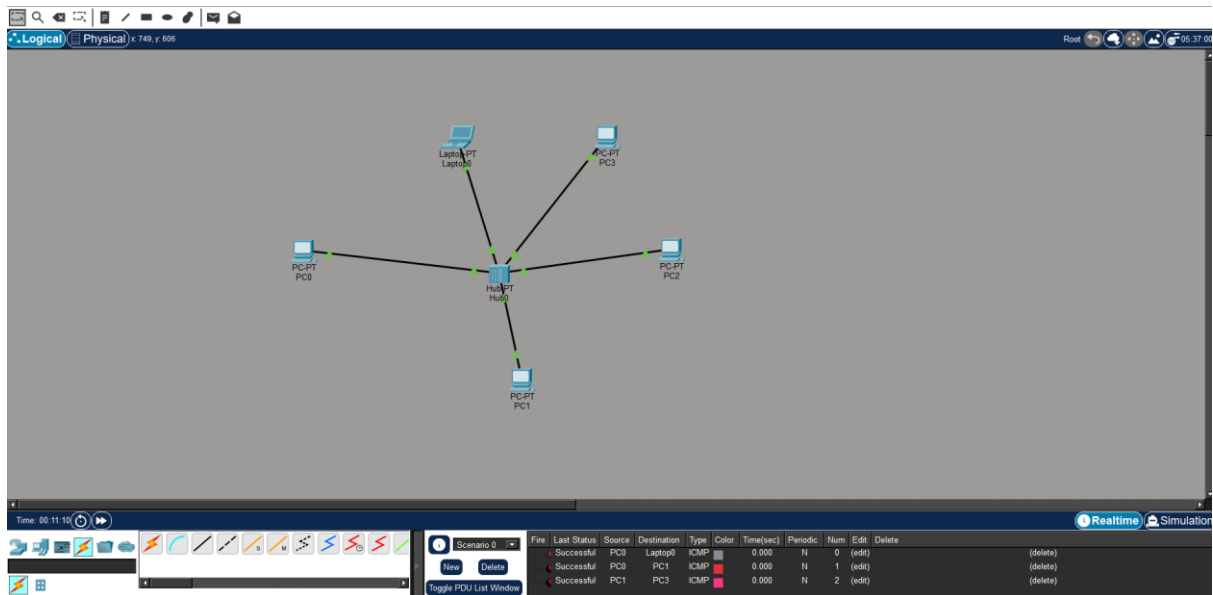
The program displays **10 meters** as the cable length for each connection.

D. Cable Length

The program uses:

Cable Length = 10 meters

DIAGRAM:



EXPLANATION:

H1 – Host 1

H1 is the first computer in the network. It has a dedicated connection to SW1 through a Cat6 UTP cable.

H2 – Host 2

H2 is the second computer. It also has an independent cable connection to SW1.

H3 – Host 3

H3 represents the third computer. Its communication with other hosts is managed through SW1.

H4 – Host 4

H4 is the fourth computer connected directly to SW1 using its own cable.

H5 – Host 5

H5 represents the fifth computer and has a separate connection to the central switch.

SW1 – Central Switch

SW1 is positioned at the center because it acts as the main connecting device. All hosts communicate through this switch.

Lines Between Hosts and SW1

The lines represent **Cat6 UTP cables**.

10 m Labels

The 10 m labels indicate the cable length between each host and SW1.

Overall Structure

The connections extend from the central switch toward each host, creating a **star-shaped structure**.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

ANSWER:

INTRODUCTION:

In Ethernet networks, multiple hosts may try to transmit data through the same communication medium. If two or more hosts transmit at the same time, their frames can interfere with each other, resulting in a **collision**.

CSMA/CD is a protocol designed to handle such collisions.

In this Python program, CSMA/CD is simulated using **four hosts connected to a central switch**. The program performs transmission attempts, detects collisions, generates random back-off intervals, and allows the hosts to retransmit after waiting.

Definition of CSMA/CD

CSMA/CD stands for:

Carrier Sense Multiple Access with Collision Detection

It is a medium-access mechanism associated with traditional shared-medium Ethernet.

CODE:

```
import random
```

```
LOG_FILE = "csma_log.txt"
```

```
hosts = ["H1", "H2", "H3", "H4"]
```

```
# One frame for each host  
pending = hosts.copy()
```

```
# Information about each host  
attempts = {h: 0 for h in hosts}  
collisions = {h: 0 for h in hosts}
```

```

backoff = {h: 0 for h in hosts}

random.seed(7)

with open(LOG_FILE, "w") as log:

    log.write("CSMA/CD SIMULATION LOG\n")
    log.write("=" * 50 + "\n")

    slot = 0

    while pending:

        # Find hosts ready to transmit
        ready = [h for h in pending if backoff[h] == 0]

        # If no host is ready, wait
        if not ready:
            slot += 1

            for h in pending:
                backoff[h] -= 1

            print("Slot", slot, ": Waiting")
            log.write(f"Slot {slot}: Waiting\n")
            continue

        slot += 1

        # Record transmission attempts
        for h in ready:
            attempts[h] += 1

        # Collision occurs if more than one host transmits
        if len(ready) > 1:

            print("Slot", slot,
                  ": COLLISION ->", ", ".join(ready))

            log.write(
                f"Slot {slot}: COLLISION -> "
                f"{', '.join(ready)}\n"
            )

            for h in ready:

                collisions[h] += 1

```

```

    # Binary exponential back-off
    k = min(collisions[h], 10)

    backoff[h] = random.randint(
        0, (2 ** k) - 1
    )

    print(
        h,
        "Attempt", attempts[h],
        "Back-off =", backoff[h]
    )

    log.write(
        f"{h}: Attempt {attempts[h]}, "
        f"Back-off = {backoff[h]}\n"
    )

else:

    # Successful transmission
    h = ready[0]

    print(
        "Slot", slot,
        ":", h,
        "transmitted successfully"
    )

    log.write(
        f"Slot {slot}: {h} transmitted "
        f"successfully "
        f"(Attempt {attempts[h]})\n"
    )

    pending.remove(h)

print("\nAll frames transmitted successfully.")
log.write("\nAll frames transmitted successfully.\n")

```

CODE OUTPUT:

Slot 1 : COLLISION -> H1, H2, H3, H4

H1 Attempt 1 Back-off = 1

H2 Attempt 1 Back-off = 0

H3 Attempt 1 Back-off = 1

H4 Attempt 1 Back-off = 0

Slot 2 : COLLISION -> H2, H4

H2 Attempt 2 Back-off = 0

H4 Attempt 2 Back-off = 0

Slot 3 : COLLISION -> H2, H4

H2 Attempt 3 Back-off = 5

H4 Attempt 3 Back-off = 0

Slot 4 : H4 transmitted successfully

Slot 5 : Waiting

Slot 6 : COLLISION -> H1, H3

H1 Attempt 2 Back-off = 1

H3 Attempt 2 Back-off = 0

Slot 7 : H3 transmitted successfully

Slot 8 : Waiting

Slot 9 : H1 transmitted successfully

Slot 10 : Waiting

Slot 11 : Waiting

Slot 12 : Waiting

Slot 13 : H2 transmitted successfully

All frames transmitted successfully.

The screenshot displays the Packet Tracer software interface. At the top is a menu bar with options: File, Edit, Options, View, Tools, Extensions, Window, Help. Below the menu is a toolbar with icons for file operations and network building. The main workspace shows a network topology with a central Hub0 connected to four PCs (PC0, PC1, PC2, PC3) in a star configuration. The interface is divided into several panels: a top status bar showing 'Logical' and 'Physical' modes; a right-hand 'Simulation Panel' with an 'Event List' table; a bottom-left 'PLAY CONTROLS' section with a timeline and play buttons; and a bottom-right 'Scenarios' and 'Fire' section. The 'Event List' table shows the following data:

Vis.	Time(sec)	Last Device
	0.000	-
	0.000	-
	0.001	PC0
	0.001	-
	0.002	PC0
	0.002	Hub0
	0.002	Hub0

The 'PLAY CONTROLS' section shows a timeline at 00:05:53.780 and a 'PLAY CONTROLS' button. The 'Scenarios' section shows a list of scenarios, and the 'Fire' section shows a table of active scenarios:

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	In Progress	PC0	PC1	ICMP		0.000	N	0	(edit)	(delete)
	In Progress	PC0	PC3	ICMP		0.000	N	1	(edit)	(delete)

- **H1, H2, H3, H4 – Hosts:** These are the four computers that generate and transmit data frames in the network.
- **SW1 – Central Switch:** SW1 connects all four hosts and forms the center of the star topology. Communication between the hosts passes through the switch.
- **Network Connections:** Each host has a separate connection to SW1, allowing the hosts to communicate through the central device.
- **Frame Transmission:** Each host attempts to transmit its frame when it is ready. The program records every transmission attempt.
- **Collision:** If two or more hosts attempt to transmit at the same time during the simulation, a collision is detected.
- **Random Back-off:** After a collision, the affected hosts wait for randomly calculated time intervals before attempting transmission again.
- **Retransmission:** Once the waiting period is completed, the hosts try to transmit their frames again until successful.
- **Log File:** Every transmission attempt, collision, and back-off interval is recorded in `csma_log.txt` for analysis

- **H1, H2, H3, H4 – Hosts:** These are the four computers that generate and transmit data frames in the network.
- **SW1 – Central Switch:** SW1 connects all four hosts and forms the center of the star topology. Communication between the hosts passes through the switch.
- **Network Connections:** Each host has a separate connection to SW1, allowing the hosts to communicate through the central device.
- **Frame Transmission:** Each host attempts to transmit its frame when it is ready. The program records every transmission attempt.
- **Collision:** If two or more hosts attempt to transmit at the same time during the simulation, a collision is detected.
- **Random Back-off:** After a collision, the affected hosts wait for randomly calculated time intervals before attempting transmission again.
- **Retransmission:** Once the waiting period is completed, the hosts try to transmit their frames again until successful.
- **Log File:** Every transmission attempt, collision, and back-off interval is recorded in `csma_log.txt` for analysis

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments

Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done
---------------------------	----	--	---------------------------------	--------------------	-----------------------